

Ribon

RNA secondary-structure visualization and analysis

v0.1.0 2026-08-13

GPL-2.0-only

RNA secondary-structure visualization and analysis

Ribon contributors

ribon visualizes and analyzes RNA and DNA secondary structures in Typst. It draws supplied extended dot-bracket structures, predicts structures from sequence alone, computes ensemble observables, and connects results directly to annotations, comparisons, and quantitative plots.

This manual covers data conventions, layout, thermodynamic models, constraints, core and advanced analyses, quantitative plots, error handling, reproducibility, validation, and the complete public API.

Table of Contents

Overview	3	VI.2 Integrated analysis	18
I.1 Architecture and responsibility		VI.3 Thermodynamic models	19
boundaries	3	VI.4 Hard, soft, and probing constraints . . .	20
I.2 Supported capabilities	3	VI.5 Specialized ensemble operations	21
Installation and first document	5	Quantitative and comparative figures	23
II.1 Typst Universe	5	VII.1 Structure comparison	23
II.2 Smallest drawing	5	VII.2 Pair-probability dot plot	23
II.3 Prediction to annotated figure	6	VII.3 Mountain plot	24
Sequences and structures	7	VII.4 Axes and plot-area geometry	25
III.1 Sequence alphabet	7	VII.5 Legend layout and sharing	26
III.2 Extended dot-bracket notation	7	VII.6 Composing alternatives	29
III.3 Structure elements	7	Advanced analyses	30
Drawing and layouts	9	VIII.1 Pseudoknot rendering	30
IV.1 Layout methods	9	VIII.2 Computational scope	31
IV.2 Geometry controls	11	API reference: protocol and models	33
IV.3 Numbering and direction	11	API reference: constraints	35
IV.4 Editable scenes	12	API reference: core analysis	36
Themes and annotations	14	API reference: advanced analysis	38
V.1 Themes	14	API reference: structure and rendering	41
V.2 Annotation constructors	14	API reference: annotations and plots	43
V.3 Continuous values and legends	16	Errors, reproducibility, and performance . . .	49
Prediction and thermodynamic analysis	18	XV.1 Error handling	49
VI.1 Stable response envelope	18		

XV.2 Determinism	49
XV.3 Performance guidance	49
Validation and reproducible export	
checklist	50
License, provenance, and citation	51
References	52
Index	54

Part I

Overview

Ribon is a Typst package for two related tasks:

- drawing an RNA or DNA secondary structure from a sequence and extended dot-bracket notation;
- computing thermodynamic and ensemble properties from a sequence, then rendering the result without a conversion step.

The public API is deliberately split into analysis, layout, rendering, annotation, and plot layers. A document can use only draw with a supplied structure, or can run `analyze` and send the returned response to `render`, `dot-plot`, or `mountain-plot`.

I.1 Architecture and responsibility boundaries

Layer	Implementation	Responsibility
Public entry	<code>lib.typ</code>	Stable imports only; no algorithm or renderer implementation.
Protocol	<code>src/protocol.typ</code>	One versioned JSON request/response envelope over a single-export WASM ABI.
Constraints	<code>src/constraints.typ</code>	Hard, soft, and probing constraint constructors.
Analysis	<code>src/analysis.typ</code>	Typed wrappers, structure decomposition, and result-derived annotation helpers.
Annotations	<code>src/annotations.typ</code>	Themes, labels, colors, legends, base/pair/region metadata.
Rendering	<code>src/render.typ</code>	Scene transformation and native Typst primitives.
Chart geometry	<code>src/chart.typ</code>	Scientific and matrix axes, data-coordinate transforms, ticks, and categorical legends.
Plots	<code>src/plots.typ</code>	RNA-specific plot data, structure differences, dot plots, and mountain plots.
Engine	Rust/WASM	Thermodynamics, dynamic programming, decoding, validation, and coordinates.

Visible output is generated with Typst `curve`, `line`, `circle`, `polygon`, `rect`, and `text` primitives. No bitmap is embedded by the renderer.

I.2 Supported capabilities

Ribon includes:

- six layouts: `naview`, `simple`, `circular`, `linear`, `turtle`, and `puzzler`
- extended dot-bracket validation, crossing pairs, strand breaks, and editable scene coordinates;
- MFE folding and energy evaluation;
- log-domain partition functions, base-pair and unpaired probabilities, centroid and MEA decoding;
- hard/soft constraints, SHAPE/DMS pseudo-energies, sampling, suboptimal enumeration, accessibility, and local analysis;
- duplex, cofold, circular RNA, comparative folding, modified bases, and G-quadruplex analysis;

- pseudoknot prediction/evaluation, conditional density-2 ensembles, exact fatgraph topology, exact landscapes, inverse design, and ligand microstates;
- continuous color tracks, motif annotations, structure comparison, probability dot plots, mountain plots, and composable multi-panel figures.

Exactness policy. An operation is never silently replaced by a smaller approximate state space. Inputs predicted to exceed document-time limits return a stable `resource_limit` error unless the caller explicitly opts in.

Part II

Installation and first document

II.1 Typst Universe

Once released, import the package by version:

```
#import "@preview/ribon:0.1.0": *
```

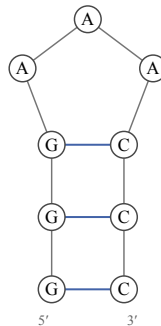
When working from a local checkout, import the package entry point:

```
#import "../package/lib.typ": *
```

The runtime consists of Typst source plus `ribon_plugin.wasm`. It does not need a Python interpreter, native shared library, shell command, or network request while compiling a document.

II.2 Smallest drawing

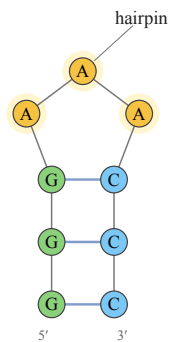
```
#import "@preview/ribon:0.1.0": draw  
  
#draw(  
  "GGGAAACCC",  
  structure: "(((...)))",  
  width: 8cm,  
  height: 4.7cm,  
)
```



Positions in every public API are one-based. Strand breaks are written as `&` in both sequence and structure and do not consume a nucleotide index.

II.3 Prediction to annotated figure

```
#import "@preview/ribon:0.1.0": *  
  
#let result = analyze("GGGAAACCC")  
  
#render(  
  result,  
  which: "mea",  
  method: "naview",  
  width: 8cm,  
  height: 4.7cm,  
  theme: varna-theme,  
  annotations: (  
    highlight(4, 6),  
    label-annotation(5, [hairpin], dx: 22pt, dy: -20pt),  
  ),  
)
```



The response retains the resolved model and constraints. Result-driven rendering therefore uses the same sequence, structure, probabilities, and execution metadata as the analysis call.

Part III

Sequences and structures

III.1 Sequence alphabet

RNA models accept canonical RNA bases. The DNA model accepts DNA input and preserves the selected parameter family. Whitespace is normalized by the engine. Use & to separate strands. Modified-nucleotide calculations receive a canonical sequence plus explicit modified-base descriptors.

III.2 Extended dot-bracket notation

Parenthesis pairs describe the planar layer. Additional matching bracket alphabets describe crossing pairs. `validate` returns a normalized structure, pair records, bracket levels, strand breaks, and sequence length.

```
#let parsed = data(validate(  
  "GCGCAAAGCGC",  
  "((([..))..]]",  
))  
  
#table(  
  columns: 2,  
  [Structure], [#raw(parsed.structure)],  
  [Base pairs], [#parsed.pairs.len()],  
)
```

Structure	((([..))..]]
Base pairs	4

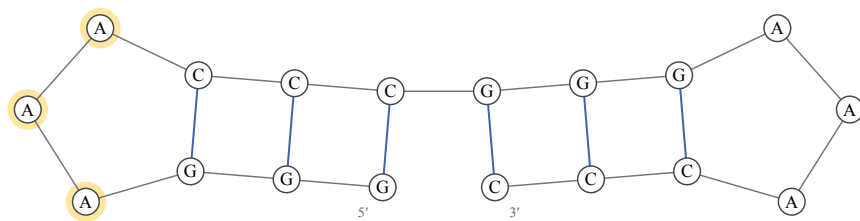
Validation first. Call `try-request` when parsing untrusted user input. It returns an error envelope instead of stopping document compilation.

III.3 Structure elements

`structure-elements` derives stems and planar hairpin, bulge, internal, multiloop, and exterior elements. Crossing pairs remain in the parsed result; motif classification uses the well-defined planar layer.

```
#let elements = structure-elements(sequence, reference)
#let hairpin = elements.loops.find(it => it.kind == "hairpin")

#draw(
  sequence,
  structure: reference,
  width: 12cm,
  height: 5cm,
  numbering: none,
  annotations: element-annotations(
    hairpin,
    fill: rgb("#ffe082").transparentize(25%),
  ),
)
```



Part IV

Drawing and layouts

IV.1 Layout methods

Method	Use	Geometry
naview	General secondary-structure diagrams and loop/stem balance.	Loop-aware planar geometry.
simple	RNAplot-like regular-bond radial layout.	Deterministic radial geometry.
circular	Crossing structures and global topology.	Bases on a circle; chords show pairs.
linear	Duplexes, strand interactions, and long sequences.	Single strands use pair arcs; multiple strands use alternating antiparallel rows, interstrand rungs, and outward intramolecular arcs.
turtle	Affine loop/stem construction.	Loop-centered recursive geometry.
puzzler	Crowded long structures.	Collision-reduced loop layout.

The layout methods expose distinct geometric models rather than cosmetic presets. `naview` follows the modified radial formulation of Brucoleri and Heinrich [1]; `puzzler` follows the collision-aware outerplanar construction introduced with RNApuzzler [2]; and `simple` follows the regular-bond radial convention used by RNAplot [3].¹

¹ViennaRNA, “RNAplot command documentation,” <https://viennarna.readthedocs.io/en/latest/man/RNAplot.html>.

```
#grid(
  columns: (1fr, 1fr),
  gutter: 4mm,
  ..("naview", "simple", "circular", "linear", "turtle", "puzzler").map(method =>
    figure(
      draw(
        "GGGAAACCC",
        structure: "(((...)))",
        method: method,
        width: 7.5cm,
        height: 3.9cm,
        numbering: none,
      ),
      caption: method,
    )
  ),
)
```

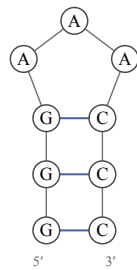


Figure 1: naview

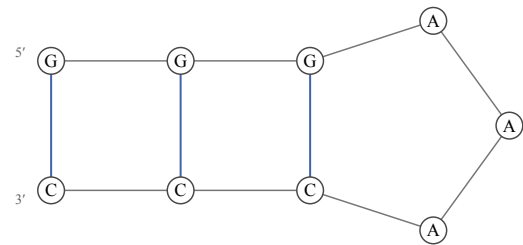


Figure 2: simple

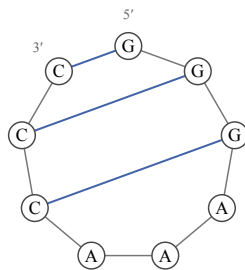


Figure 3: circular

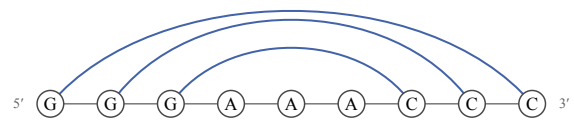


Figure 4: linear

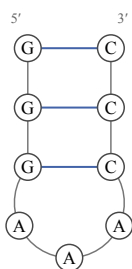


Figure 5: turtle

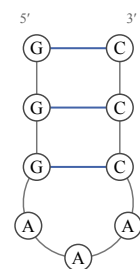


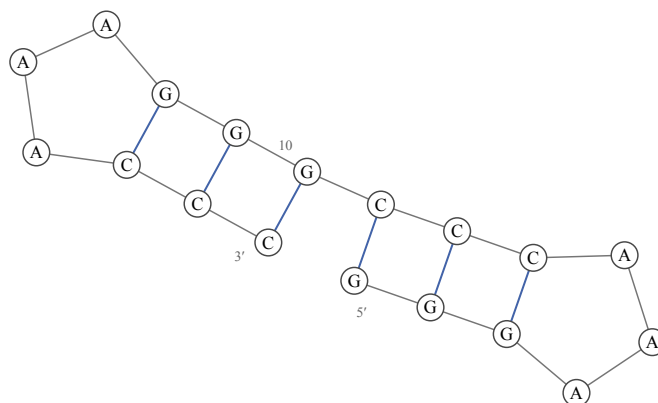
Figure 6: puzzler

IV.2 Geometry controls

draw, render, and render-scene share figure controls:

- width (default: 10cm): Target width.
- height (default: auto): Optional target height.
- fit (default: "contain"): Preserves the natural aspect ratio inside the target box.
- rotation (default: 0deg): Rotates coordinates before fitting.
- mirror-x / mirror-y (default: false): Reflects coordinates before fitting.
- clip (default: false): Clips geometry to the requested box when enabled.
- detail (default: auto): Controls long-sequence level of detail.
- node-radius (default: 5pt): Nucleotide circle radius.
- font-size (default: 6.8pt): Nucleotide label size.

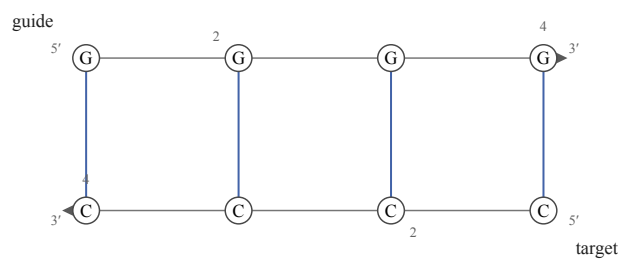
```
#draw(
  sequence,
  structure: reference,
  width: 12cm,
  height: 6cm,
  rotation: 24deg,
  mirror-x: true,
  fit: "contain",
)
```



IV.3 Numbering and direction

numbering-style supports fixed intervals, explicit positions, per-strand numbering, first/last labels, size, and fill. The renderer measures labels and chooses outward anchors. Terminus and direction indicators use separate controls.

```
#draw(
  "GGGG&CCCC",
  structure: "((((&))))",
  method: "linear",
  width: 12.5cm,
  height: 4.8cm,
  show-direction: true,
  numbering: numbering-style(every: 2, per-strand: true),
  annotations: (
    strand-label(1, [guide], dx: -20pt, dy: -14pt, leader: false),
    strand-label(2, [target], dx: 20pt, dy: 14pt, leader: false),
  ),
)
```



IV.4 Editable scenes

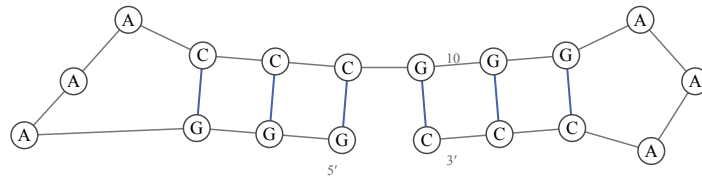
layout returns normalized coordinates without drawing them. Modify points and pass the dictionary or original response to render-scene.

```

#let scene = data(layout(sequence, reference))
#let moved = scene.points.enumerate().map((i, point)) => {
  if i == 3 { (x: point.x - 0.15, y: point.y - 0.12) }
  else { point }
})
#scene.insert("points", moved)

#render-scene(scene, width: 10cm, height: 5cm)

```



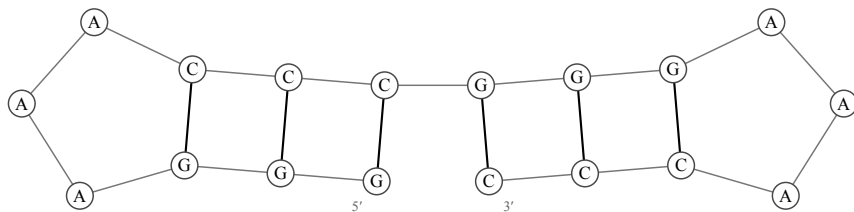
Part V

Themes and annotations

V.1 Themes

default-theme is restrained and print-oriented. varna-theme adds a nucleotide palette informed by the visual conventions of VARNA [4]. A theme is a Typst dictionary, so project-specific overrides are ordinary dictionary addition.

```
#let journal-theme = default-theme + (  
  pair-stroke: (paint: black, thickness: 0.8pt),  
  node-fill: white,  
  text-fill: black,  
  node-text-contrast: "fixed",  
)  
  
#draw(  
  sequence,  
  structure: reference,  
  width: 12cm,  
  height: 5cm,  
  theme: journal-theme,  
  numbering: none,  
)
```



V.2 Annotation constructors

Constructor	Purpose
highlight(from, to)	Background region for one contiguous range.
highlight-positions(positions)	Background region for an arbitrary set.
base-annotation(at)	Base fill, stroke, text color, and per-node WCAG contrast policy.
pair-annotation(i, j)	Base-pair edge override.

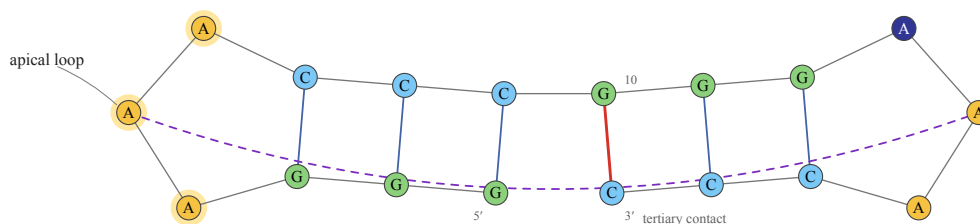
Constructor	Purpose
<code>interaction-annotation(i, j)</code>	Non-secondary interaction with bend and optional label.
<code>coaxial-annotation(...)</code>	Coaxially stacked helices.
<code>label-annotation(at, body)</code>	Measured free-form label with optional leader.
<code>strand-label(strand, body)</code>	Start/end label for a strand.

Labels are kept on one line by default. For a hand-finished figure, use `dx` and `dy` to set the label center relative to its nucleotide; `leader-bend`, `leader-stroke`, and the two `leader-gap` values adjust the connector independently. Set `width` only when intentional wrapping is wanted, and use the `box-*` values for a background or outline. Interaction labels use `label-position` to move along the curve and `label-dx`/`label-dy` for the final offset. Strand labels expose the same placement controls as nucleotide labels.

Nucleotide text defaults to `node-text-contrast: "aa"`. For every solid node fill, the renderer computes WCAG 2.2 relative luminance and chooses black or white, whichever yields the larger contrast ratio; this guarantees the 4.5:1 threshold for small text. "aaa" requires 7:1 and reports the nucleotide index, fill, and best achievable ratio when neither black nor white can satisfy it.² `node-text-contrast: "fixed"` retains `theme.text-fill`. A per-node `text-contrast` overrides the global policy, while an explicit `text-fill` has highest priority. Transparent fills require `contrast-background` so the effective color can be computed; gradients and patterns require an explicit `text-fill`. `contrast-on-failure: "best"` is an explicit opt-out from strict AAA failure.

²W3C, "WCAG 2.2 Success Criterion 1.4.3: Contrast (Minimum)," <https://www.w3.org/TR/WCAG22/#contrast-minimum>. W3C, "Success Criterion 1.4.6: Contrast (Enhanced)," <https://www.w3.org/TR/WCAG22/#contrast-enhanced>. W3C, "Understanding relative luminance," <https://www.w3.org/WAI/WCAG21/Understanding/relative-luminance.html>. W3C, "Guidance on Applying WCAG 2.2 to Non-Web Information and Communications Technologies," <https://www.w3.org/TR/wcag2ict-22/>.

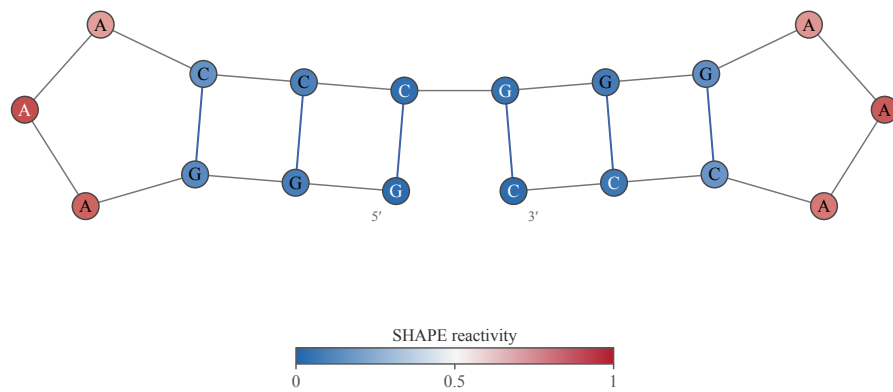
```
#draw(
  sequence,
  structure: reference,
  width: 14.5cm,
  height: 6.1cm,
  theme: varna-theme,
  annotations: (
    highlight(4, 6, fill: rgb("#ffe082").transparentize(25%)),
    base-annotation(13, fill: rgb("#313695"), text-contrast: "aaa"),
    pair-annotation(10, 18, stroke: (paint: rgb("#d73027"), thickness: 1.2pt)),
    label-annotation(5, [apical loop], dx: -34pt, dy: -22pt, leader-bend: 0.08),
    interaction-annotation(
      5,
      14,
      label: [tertiary contact],
      label-position: 0.62,
      label-dx: 12pt,
      label-dy: 14pt,
    ),
  ),
),
)
```



V.3 Continuous values and legends

color-scale defines one reusable numerical mapping. value-annotations retains that scale with its base annotations, so draw and render generate an exact matching legend automatically. Pass legend: false to the renderer to hide it or a legend-style value to control placement and framing. color-legend and with-legend remain available for legends shared by several figures; suppress the local legend on each component drawing before composing the shared one. reactivity-annotations, accessibility-annotations, local-accessibility-annotations, and entropy-annotations create the same metadata-bearing tracks.


```
#let scale = color-scale(  
  minimum: 0,  
  maximum: 1,  
  label: [SHAPE reactivity],  
)  
  
#draw(  
  sequence,  
  structure: reference,  
  width: 12.5cm,  
  height: 5cm,  
  annotations: value-annotations(  
    values,  
    scale: scale,  
    legend: (width: 4.2cm),  
  ),  
  numbering: none,  
  legend: legend-style(stroke: none),  
)
```



Part VI

Prediction and thermodynamic analysis

VI.1 Stable response envelope

Every engine call uses `ribon.analysis/1`. Successful responses contain `ok`, `engine`, the resolved model, resolved constraints, execution metadata, and `result.(kind, data)`. Use `data(response)` to extract the typed payload.

```
#let response = analyze("GGGAAACCC")
#let result = data(response)

#table(
  columns: 2,
  [MFE], [#raw(result.mfe_structure)],
  [Energy], [#calc.round(result.mfe_energy_kcal_mol, digits: 1) kcal/mol],
  [Centroid], [#raw(result.centroid_structure)],
  [MEA], [#raw(result.mea_structure)],
  [Pair records], [#result.pair_probabilities.len()],
)
```

MFE	(((...)))
Energy	−1.2 kcal/mol
Centroid	(((...)))
MEA	(((...)))
Pair records	9

VI.2 Integrated analysis

`analyze` combines thermodynamic dynamic programming for minimum-free-energy folding [5] with the equilibrium partition function and pair marginals [6]. It then decodes the same probability matrix as a generalized centroid structure [7] and a maximum expected accuracy structure [8], returning ensemble free energy and entropy summaries in the same request.

```

#let                                     decoder-sequence
= "GCGCCUUGAAAGUCCAGAGGACUUGGUUUUAUUGGGUAGUUGAGGUUGGCCCAUCUC"
#let result = analyze(decoder-sequence)

#grid(
  columns: (1fr, 1fr, 1fr),
  gutter: 3mm,
  ..(("MFE", "mfe"), ("Centroid", "centroid"), ("MEA", "mea")).map(((title, which))
=> figure(
  render(
    result,
    which: which,
    width: 5.1cm,
    height: 4cm,
    node-radius: 1.8pt,
    font-size: 2.6pt,
    detail: "full",
    numbering: none,
    show-ends: false,
  ),
  caption: title,
),
)

```

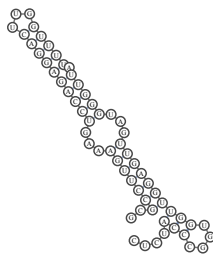


Figure 7: MFE

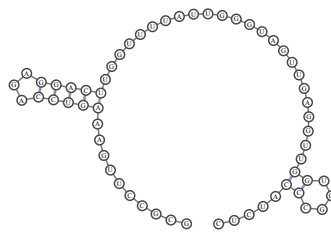


Figure 8: Centroid

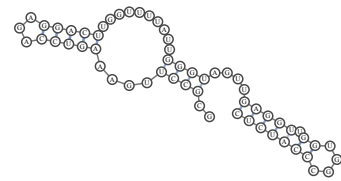


Figure 9: MEA

VI.3 Thermodynamic models

`analysis-model` constructs an immutable nearest-neighbor model descriptor. Temperature, minimum loop length, dangle treatment, monovalent salt concentration, and MEA gamma are explicit. The bundled RNA tables use the Turner 2004 family described by Mathews et al. [9], while `dna-model` selects the corresponding Mathews 2004 DNA parameter family. Non-reference monovalent salt concentrations apply the loop, stack, multiloop, and duplex-initiation correction model of Yao et al. [10]. `custom-model` applies a normalized, fingerprinted table overlay to an RNA or DNA base family.³

³University of Rochester, "Nearest Neighbor Database," <https://rna2.urmc.rochester.edu/NNDB/>. University of Rochester, "Nearest Neighbor Database: Download," <https://rna2.urmc.rochester.edu/NNDB/download.html>.

```

#let physiological = analysis-model(
  temperature: 37,
  min-loop: 3,
  dangles: 2,
  salt: 0.15,
  mea-gamma: 1,
)

#let dna = dna-model(temperature: 25, salt: 0.1)

#table(
  columns: 3,
  [Model], [Temperature], [Salt],
    [#physiological.id],      [#physiological.temperature_celsius °C],
[#physiological.salt_molar M],
  [#dna.id], [#dna.temperature_celsius °C], [#dna.salt_molar M],
)

```

Model	Temperature	Salt
ribon-turner-2004	37 °C	0.15 M
ribon-mathews-dna-2004	25 °C	0.1 M

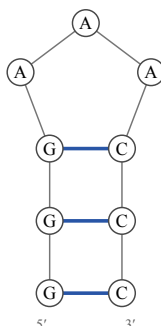
Model provenance and table SHA-256 fingerprints are returned by parameters.

VI.4 Hard, soft, and probing constraints

Probing reactivities can be converted either to paired/stacked pseudo-energies using the SHAPE-directed model of Deigan et al. [11] or to probabilistic soft constraints using the mapping of Zarringhalam et al. [12]. Explicit hard constraints and direct per-position or per-pair energies remain available independently of either conversion.

```
#let constraints = folding-constraints(
  force-unpaired: (4, 5, 6),
  force-pairs: (constraint-pair(1, 9)),
  soft: soft-constraints(
    unpaired: (position-energy(5, -0.8)),
    pairs: (pair-energy(1, 9, -1.2)),
  ),
  probing: probing-data(
    (0.1, 0.2, 0.8, 0.9, 0.7, 0.2, 0.1, 0.1, 0.0),
    kind: "shape",
    method: "deigan",
  ),
)

#render(
  analyze("GGAAACCC", constraints: constraints),
  width: 8cm,
  height: 4.7cm,
  numbering: none,
)
```



The same constraint object can be passed to folding, partition, sampling, suboptimal, accessibility, and fixed-structure evaluation operations.

VI.5 Specialized ensemble operations

Operation	Purpose
fold	MFE only.
evaluate	Energy and decomposition for a supplied planar structure.
sample	Reproducible stochastic backtracking with explicit seed.
suboptimal	Deterministic k-best structures within an energy band.
accessibility	Exact joint-unpaired probabilities and opening energies for requested windows.
local	Sliding-window pair and accessibility probabilities.
circular	Circular-RNA MFE and ensemble analysis.
duplex	Connected intermolecular duplex ensemble without intramolecular pairs.

Operation	Purpose
cofold	Unrestricted two-strand ensemble and optional mass-action concentrations.

Energy-band enumeration follows the complete suboptimal-folding formulation of Wuchty et al. [13]. Local pair probabilities follow the window-averaged ensemble construction of Bernhart et al. [14], while heterodimer partition functions and pair probabilities follow the cofold formulation of Bernhart et al. [15].

Part VII

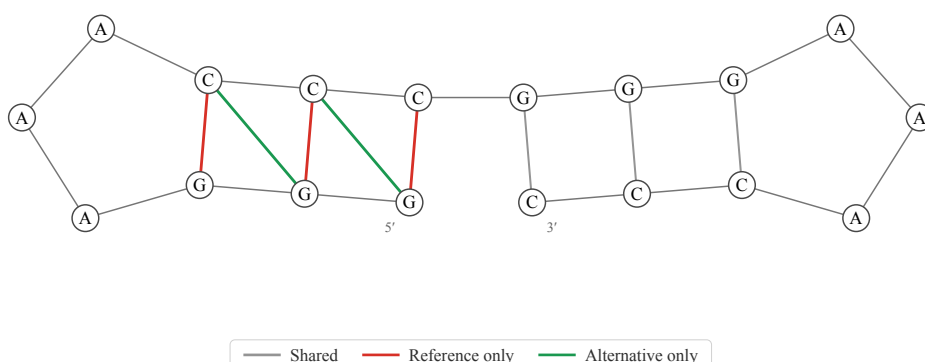
Quantitative and comparative figures

Quantitative figures separate data, coordinate transforms, plot-area geometry, legend composition, and visual styling. This keeps values invariant while the figure is adapted to a page, column, panel grid, or shared legend.

VII.1 Structure comparison

`structure-difference` returns shared, reference-only, and alternative-only pair arrays. `compare-structures` displays all three classes on reference coordinates.

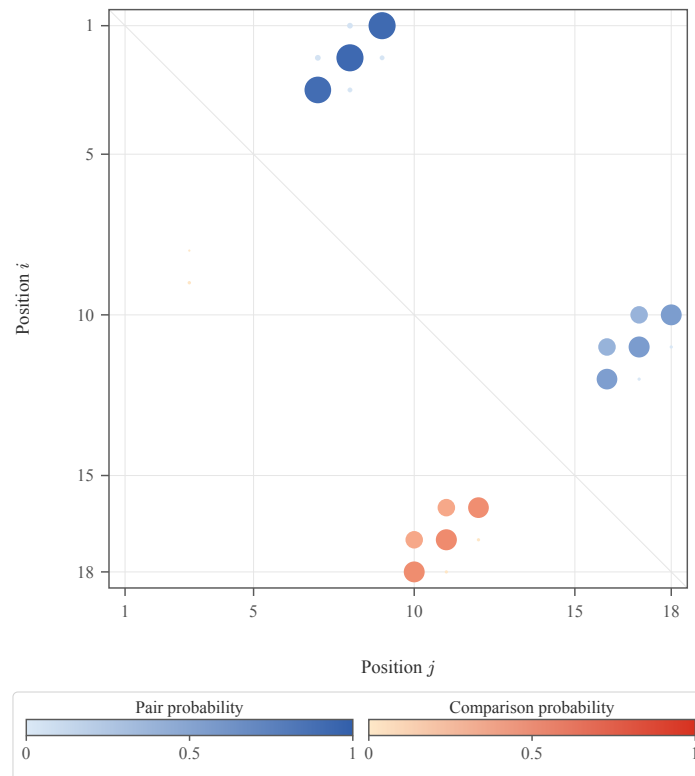
```
#compare-structures(  
  sequence,  
  reference,  
  alternative,  
  width: 13cm,  
  height: 5.3cm,  
  numbering: none,  
)
```



VII.2 Pair-probability dot plot

The plotted matrix contains McCaskill base-pair marginals [6]. The upper triangle shows the primary ensemble. A second ensemble may be placed in the lower triangle. Without a comparison, the lower triangle marks pairs in a reference or inferred MFE structure.

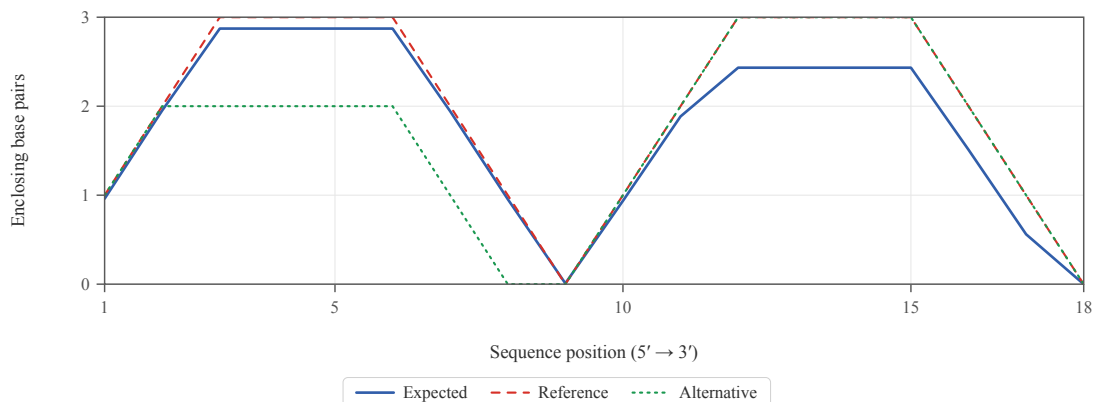
```
#dot-plot(
  sequence,
  probabilities: prediction,
  comparison: perturbed,
  width: 9.2cm,
  threshold: 0.005,
)
```



VII.3 Mountain plot

`mountain-plot` uses the mountain representation introduced by Hogeweg and Hesper [16]. It draws the expected number of enclosing pairs at each position and can overlay discrete profiles from one or more supplied structures. `mountain-profile` returns those exact numerical series when they are needed for a table, assertion, or another visualization.


```
#mountain-plot(
  sequence,
  probabilities: prediction,
  reference-structures: (
    (label: [Reference], structure: reference),
    (label: [Alternative], structure: alternative),
  ),
  width: 14.5cm,
  height: 4.8cm,
)
```

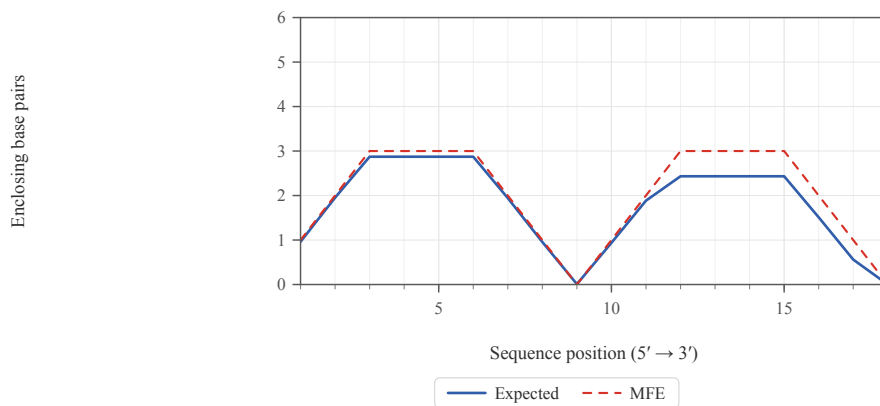


VII.4 Axes and plot-area geometry

`axis-style` independently configures each axis. Domains may ascend or descend; transforms may be linear or logarithmic; major and minor ticks may be generated from a step or supplied as values and labels. Formatters accept a function or the built-in "scientific" formatter. Grid visibility, line visibility, tick visibility, and axis labels are independent.

`plot-layout` controls the data viewport rather than its data. It accepts automatic, uniform, or per-side padding, an optional width-to-height aspect ratio, a closed frame or independent axis lines, and clipping at the viewport boundary.

```
#mountain-plot(
  sequence,
  probabilities: prediction,
  width: 14.5cm,
  height: 4.8cm,
  x-axis: axis-style(
    domain: (1, 18),
    tick-step: 5,
    minor-tick-step: 1,
    grid: "both",
  ),
  y-axis: axis-style(domain: (0, 6), tick-step: 1),
  layout: plot-layout(aspect: 2.2),
)
```

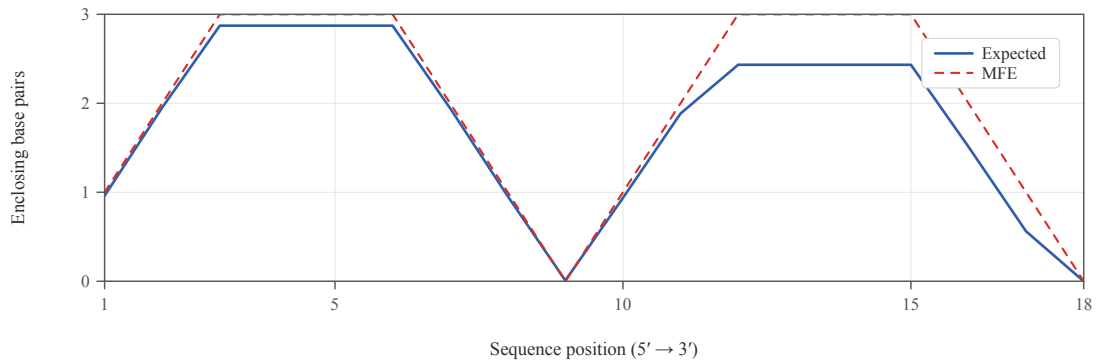


A descending domain reverses an axis. `mode: "log"` selects a positive logarithmic domain. `mountain-plot` also accepts `x2-axis` and `y2-axis` each supplied reference series can select a pair through axes: (`"x"`, `"y2"`). The expected series uses expected-axes.

VII.5 Legend layout and sharing

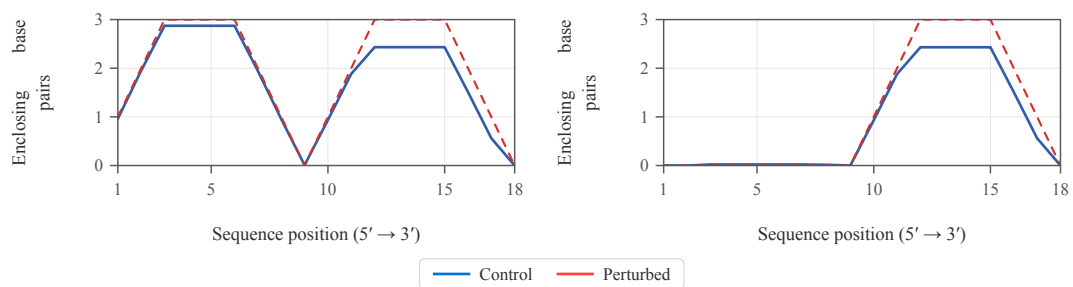
`legend-style` places a legend on top, bottom, left, or right at any of nine inner-* anchors; or at an explicit (`x`, `y`) ratio inside the data viewport. `anchor` and `offset` refine placement. `direction`, `columns`, `max-columns`, `width`, and `spacing` fields control flow without changing plot geometry.

```
#mountain-plot(
  sequence,
  probabilities: prediction,
  width: 14.5cm,
  height: 4.8cm,
  legend: legend-style(
    position: "inner-north-east",
    direction: "column",
    offset: (-3pt, 3pt),
  ),
)
```



The `legend` argument accepts `true`, `false`, or a `legend-style` dictionary. `plot-legend` creates a standalone categorical legend; `place-legend` composes it around multiple plots. `legend-panel` accepts arbitrary Typst content for a custom shared legend.

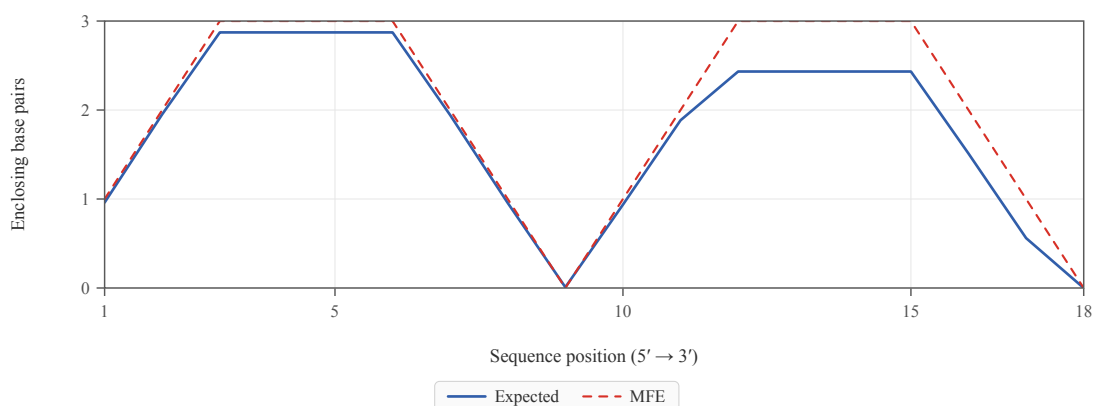
```
#place-legend(
  grid(
    columns: (1fr, 1fr),
    mountain-plot(sequence, probabilities: prediction, width: 6.8cm, height: 3.2cm,
legend: false),
    mountain-plot(sequence, probabilities: perturbed, width: 6.8cm, height: 3.2cm,
legend: false),
  ),
  plot-legend((
    legend-item([Control], stroke: blue + 1pt),
    legend-item([Perturbed], stroke: red + 1pt),
  )),
  style: legend-style(position: "bottom", columns: 2),
)
```



The visual theme remains independent of both geometry and placement:

```
#let quantitative = plot-theme(legend-fill: luma(98%))

#mountain-plot(
  sequence,
  probabilities: prediction,
  width: 14.5cm,
  height: 4.8cm,
  theme: quantitative,
)
```



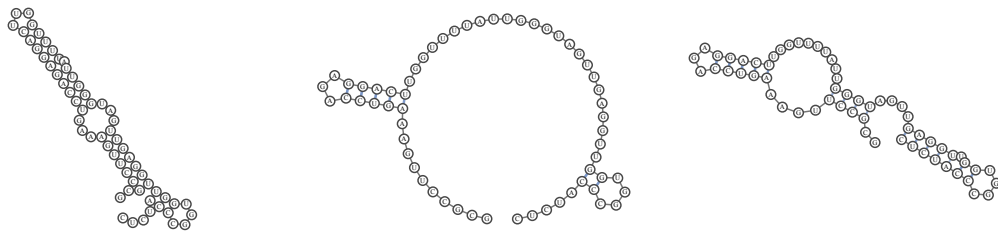
VII.6 Composing alternatives

Ribon returns each decoder, sample, and suboptimal structure through `render`. Use ordinary Typst layout primitives when several results belong in one figure.

```
#let
= "GCGCCUUGAAAGUCCAGAGGACUUGGUUUUAUUGGGUAGUUGAGGUUGGUGGCCCAUCUC"
#let decoder-prediction = analyze(decoder-sequence)

#grid(
  columns: 3,
  gutter: 3mm,
  ..("mfe", "centroid", "mea").map(which => render(
    decoder-prediction,
    which: which,
    width: 4.9cm,
    height: 3.7cm,
    node-radius: 1.8pt,
    font-size: 2.6pt,
    detail: "full",
    numbering: none,
    show-ends: false,
  )),
)
```

decoder-sequence



Part VIII

Advanced analyses

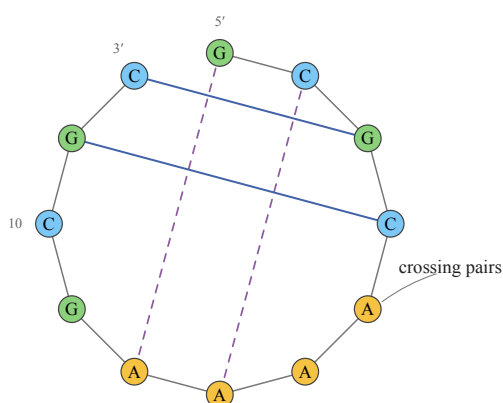
Advanced operations use the same response envelope and renderer connection as the core folding API.

Operation	Result and use
modified	Measured nearest-neighbor terms for m6A [17], pseudouridine [18], inosine [19], 7-deaza-adenosine [20], and purine/nebularine [21]; plus the model-based dihydrouridine stack penalty motivated by conformational measurements [22] and the sparse-parameter framework [23].
gquad	Integrated secondary-structure/G-quadruplex ensemble following the 2D-meets-4G model [24].
comparative	Gap-aware aligned-sequence folding with covariation pseudo-energies [25].
pseudoknot	Probability-directed ProbKnot decoding [26], restricted H-type components, Hungarian-method matching decoders [27], and opt-in arbitrary matching.
conditional-density2	Fixed-seed density-2 ensembles based on hierarchical folding [28], DP09 energies [29], and the CParty conditional decomposition [30].
fatgraph-topology	Orientable genus, boundary components, Euler characteristic, and crossing components under the fatgraph classification [31].
landscape	Globally minimum-saddle path over the complete planar structure graph using minimax Dijkstra search [32].
inverse-design	Exhaustive IUPAC-template design ranked by exact target ensemble probability.
ligand	Joint structure/ligand microstate ensemble with concentration-dependent chemical potential.

VIII.1 Pseudoknot rendering

Extended dot-bracket structures can be drawn directly. Crossing-component annotations from fatgraph-topology give each interacting component a consistent edge color.

```
#draw(
  "GCGCAAAGCGC",
  structure: "([([..))..])",
  method: "circular",
  width: 9cm,
  height: 6.8cm,
  theme: varna-theme,
  annotations: (
    label-annotation(5, [crossing pairs], dx: 31pt, dy: -16pt, leader-bend: -0.08),
  ),
)
```



VIII.2 Computational scope

landscape, inverse-design, arbitrary matching, and some nonlocal dangle variants have intrinsically exponential state spaces. The default execution policy guards document compilation. Opt in only after bounding the input:

```
#let exact = execution-policy(allow-expensive: true)

#let result = data(landscape(
  "GGGAAACCC",
  ". . . . .",
  "(((...)))",
  execution: exact,
))

#table(
  columns: 2,
  [Complete states], [#result.state_count],
  [Path steps], [#result.path.len()],
  [Saddle energy], [#result.saddle_energy_kcal_mol kcal/mol],
)
```

Complete states	20
Path steps	4
Saddle energy	3.2 kcal/mol

Part IX

API reference: protocol and models

#execution-policy

```
execution-policy(allow-expensive: false)
```

Constructs document-time resource policy. The default returns `resource_limit` for guarded requests.

#analysis-model

```
analysis-model(id: "ribon-turner-2004", temperature: 37.0, min-loop: 3, dangles: 2, salt: 1.021, mea-gamma: 1.0)
```

Constructs the RNA model descriptor shared across operations.

#dna-model

```
dna-model(temperature: 37.0, min-loop: 3, dangles: 2, salt: 1.021, mea-gamma: 1.0)
```

Constructs the Mathews 2004 DNA model descriptor.

#thermodynamic-parameter-overrides

```
thermodynamic-parameter-overrides(name, fingerprint-sha256, tables: (:))
```

Builds a normalized custom-table overlay. Omitted tables inherit the selected base family.

#custom-model

```
custom-model(parameter-overrides, base: "rna", temperature: 37.0, min-loop: 3, dangles: 2, salt: 1.021, mea-gamma: 1.0)
```

Constructs a model backed by a validated overlay.

#try-request

```
try-request(operation, input, model: analysis-model(), constraints: none, options: (:), execution: execution-policy(), id: none)
```

Runs one raw protocol operation and always returns an envelope. Inspect `ok` and `error`. (`code`, `message`) for invalid input.

#request

```
request(operation, input, model: analysis-model(), constraints: none, options: (:), execution: execution-policy(), id: none)
```

Runs one raw protocol operation and panics with a stable error code when the response is unsuccessful.

IX API reference: protocol and models

#**data**

```
data(response)
```

Validates a successful analysis/1 response and extracts `result.data`.

Part X

API reference: constraints

#constraint-pair

```
constraint-pair(i, j)
```

Creates a one-based pair entry for hard constraints.

#position-energy

```
position-energy(position, energy)
```

Creates a position-specific pseudo-energy in kcal/mol.

#pair-energy

```
pair-energy(i, j, energy)
```

Creates a pair-specific pseudo-energy in kcal/mol.

#soft-constraints

```
soft-constraints(unpaired: (), paired: (), pairs: (), stack: ())
```

Builds generic unpaired, paired, pair, and stack energy terms.

#probing-data

```
probing-data(reactivities, kind: "shape", method: "deigan", slope: 1.8,  
intercept: -0.6, beta: 0.89, conversion: "0", default-probability: 0.5)
```

Builds SHAPE or DMS input. Null and negative values are treated as missing.

#folding-constraints

```
folding-constraints(force-unpaired: (), force-paired: (), force-pairs: (),  
forbid-pairs: (), max-span: none, no-gu: false, no-lonely-pairs: false, soft:  
soft-constraints(), probing: none)
```

Builds the common hard/soft/probing constraint dictionary.

Part XI

API reference: core analysis

#capabilities

```
capabilities()
```

Returns supported operations, layouts, and decoders.

#parameters

```
parameters()
```

Returns parameter metadata and SHA-256 fingerprints.

#validate

```
validate(sequence, structure, execution: execution-policy(), id: none)
```

Parses and validates extended dot-bracket notation.

#analyze

```
analyze(sequence, model: analysis-model(), constraints: none, execution:  
execution-policy(), id: none)
```

Runs integrated MFE, partition, probability, centroid, MEA, and ensemble analysis.

#fold

```
fold(sequence, model: analysis-model(), constraints: none, execution: execution-  
policy(), id: none)
```

Predicts only the MFE structure.

#evaluate

```
evaluate(sequence, structure, model: analysis-model(), constraints: none,  
execution: execution-policy(), id: none)
```

Evaluates a supplied planar structure.

#sample

```
sample(sequence, count: 100, seed: 0, unique: false, model: analysis-model(),  
constraints: none, execution: execution-policy(), id: none)
```

Performs reproducible stochastic backtracking.

#suboptimal

```
suboptimal(sequence, energy-band: 5.0, limit: 100, model: analysis-model(),  
constraints: none, execution: execution-policy(), id: none)
```

Returns deterministic k-best structures within an energy band.

#accessibility-window

```
accessibility-window(from, to)
```

Constructs a one-based inclusive accessibility interval.

#accessibility

```
accessibility(sequence, windows, model: analysis-model(), constraints: none,  
execution: execution-policy(), id: none)
```

Computes joint-unpaired probabilities and opening energies.

#local

```
local(sequence, window-size: 150, max-pair-span: 100, max-unpaired: 30, model:  
analysis-model(), execution: execution-policy(), id: none)
```

Computes sliding-window pair and accessibility probabilities.

#circular

```
circular(sequence, model: analysis-model(dangles: 0), constraints: none,  
execution: execution-policy(), id: none)
```

Analyzes a circular RNA ensemble.

#duplex

```
duplex(sequence-a, sequence-b, model: analysis-model(), execution: execution-  
policy(), id: none)
```

Computes the connected intermolecular duplex ensemble.

#cofold

```
cofold(sequence-a, sequence-b, concentration-a: none, concentration-b: none,  
model: analysis-model(), execution: execution-policy(), id: none)
```

Computes the unrestricted two-strand ensemble and optional mass-action solution.

Part XII

API reference: advanced analysis

#modified-base

```
modified-base(position, symbol, canonical-base, kind: none, paired-energy: 0.0,
unpaired-energy: 0.0, stack-energy: 0.0)
```

Describes one modified nucleotide and optional calibrated family.

#modified

```
modified(sequence, modifications, model: analysis-model(), execution: execution-
policy(), id: none)
```

Analyzes a canonical sequence with position-specific modification effects.

#gquad

```
gquad(sequence, model: analysis-model(), execution: execution-policy(), id: none)
```

Analyzes the integrated G-quadruplex/secondary-structure ensemble.

#comparative-options

```
comparative-options(covariance-weight: 1.0, incompatible-penalty: 1.0, minimum-
pair-occupancy: 0.5)
```

Configures alignment covariation scoring.

#comparative

```
comparative(alignment, options: comparative-options(), model: analysis-model(),
execution: execution-policy(), id: none)
```

Folds a gap-aware alignment.

#pseudoknot-options

```
pseudoknot-options(threshold: 0.0, iterations: 1, min-helix: 3, gamma: 1.0,
initiation: -1.38, crossing: 2.46, unpaired: 0.06, evidence-weight: 0.0, max-
components: none, max-ensemble-states: none, exact-arbitrary-ensemble: false)
```

Configures pseudoknot decoding and ensemble scope.

#pseudoknot

```
pseudoknot(sequence, options: pseudoknot-options(), model: analysis-model(),
execution: execution-policy(), id: none)
```

Predicts crossing pairs as extended dot-bracket notation.

#evaluate-pseudoknot

```
evaluate-pseudoknot(sequence, structure, options: pseudoknot-options(), model:
analysis-model(), execution: execution-policy(), id: none)
```

Evaluates a supplied extended-dot-bracket matching.

#conditional-density2-options

```
conditional-density2-options(gamma: 1.0, pk-only: false, ..energy-coefficients)
```

Constructs fixed-seed density-2 coefficients. See the source signature for all explicit DP09 terms.

#conditional-density2

```
conditional-density2(sequence, structure, options: conditional-density2-
options(), model: analysis-model(), constraints: none, execution: execution-
policy(), id: none)
```

Computes the complete fixed-seed density-2 ensemble.

#conditional-density2-sample

```
conditional-density2-sample(sequence, structure, count: 100, seed: 0, unique:
false, options: conditional-density2-options(), model: analysis-model(),
constraints: none, execution: execution-policy(), id: none)
```

Samples exact Boltzmann structures from the fixed-seed ensemble.

#conditional-density2-suboptimal

```
conditional-density2-suboptimal(sequence, structure, energy-band: 5.0, limit:
100, options: conditional-density2-options(), model: analysis-model(),
constraints: none, execution: execution-policy(), id: none)
```

Returns the lowest-energy fixed-seed extensions.

#evaluate-conditional-density2

```
evaluate-conditional-density2(sequence, seed-structure, added-structure, options:
conditional-density2-options(), model: analysis-model(), constraints: none,
execution: execution-policy(), id: none)
```

Evaluates an explicit pair of planar layers.

#fatgraph-topology

```
fatgraph-topology(sequence, structure, execution: execution-policy(), id: none)
```

Computes exact orientable fatgraph invariants and crossing components.

#**landscape**

```
landscape(sequence, start-structure, target-structure, model: analysis-model(),  
constraints: none, execution: execution-policy(), id: none)
```

Computes the globally minimum-saddle path over the planar structure graph.

#**inverse-design**

```
inverse-design(target-structure, template: none, minimum-gc-fraction: 0.0,  
maximum-gc-fraction: 1.0, return-count: 10, model: analysis-model(), constraints:  
none, execution: execution-policy(), id: none)
```

Exhaustively designs compatible sequences from an IUPAC template.

#**ligand-motif**

```
ligand-motif(id, start, sequence, structure, standard-binding-energy,  
concentration: 1.0)
```

Defines a concentration-dependent ligand-bound motif.

#**ligand**

```
ligand(sequence, motifs, model: analysis-model(), constraints: none, execution:  
execution-policy(), id: none)
```

Computes the joint structure/ligand microstate ensemble.

Part XIII

API reference: structure and rendering

#structure-elements

```
structure-elements(sequence, structure, execution: execution-policy())
```

Identifies planar stems, loop motifs, and exterior segments.

#element-annotations

```
element-annotations(element, fill: auto, stroke: auto, radius: auto, pair-stroke: auto)
```

Converts a structure element to renderer layers.

#Layout

```
layout(sequence, structure, method: "naview", execution: execution-policy(), id: none)
```

Returns normalized scene geometry without rendering.

#draw

```
draw(sequence, structure: auto, model: analysis-model(), constraints: none, method: "naview", execution: execution-policy(), ..render-arguments)
```

Draws a supplied structure or predicts and draws the MFE when structure is auto. Metadata-bearing annotation tracks generate their legends automatically; pass `legend: false` or a `legend-style` through the render arguments.

#render-scene

```
render-scene(scene, ..render-arguments)
```

Renders layout geometry, including user-edited coordinates and automatic annotation-track legends.

#render

```
render(response, which: "mfe", item: 1, method: "naview", width: 10cm, height: auto, node-radius: 5pt, font-size: 6.8pt, theme: default-theme, node-text-contrast: auto, node-contrast-background: auto, node-contrast-on-failure: auto, annotations: (), legend: auto, legend-theme: plot-theme(), probabilities: auto, fit: "contain", rotation: 0deg, mirror-x: false, mirror-y: false, clip: false, detail: auto, label-padding: auto, numbering: 10, start-number: 1, show-ends: true, show-direction: false, show-backbone: true, show-pairs: true, show-nucleotides: true)
```

XIII API reference: structure and rendering

Renders a supported protocol response as native Typst vectors. Annotation-track legends are generated automatically; `legend` accepts `auto`, a `Boolean`, or `legend-style`. Node contrast arguments override the corresponding theme values.

Part XIV

API reference: annotations and plots

#highlight

```
highlight(from, to, fill: auto, stroke: auto, radius: auto)
```

Builds a contiguous region annotation.

#highlight-positions

```
highlight-positions(positions, fill: auto, stroke: auto, radius: auto)
```

Builds an arbitrary-position region annotation.

#base-annotation

```
base-annotation(at, fill: auto, stroke: auto, text-fill: auto, text-contrast: auto, contrast-background: auto, contrast-on-failure: auto)
```

Overrides one nucleotide. `text-fill` takes precedence over the per-node `text-contrast` policy; transparent fills can name the opaque color behind the node.

#pair-annotation

```
pair-annotation(i, j, stroke: auto)
```

Overrides one pair edge.

#interaction-annotation

```
interaction-annotation(i, j, stroke: auto, bend: 0.18, label: none, label-position: 0.5, label-dx: 0pt, label-dy: 0pt, label-fill: auto, label-size: auto, label-width: auto, label-align: center, label-box-fill: auto, label-box-stroke: none, label-box-inset: 1pt, label-box-radius: 1pt)
```

Adds a non-secondary interaction. The label can be moved along the curve and offset independently.

#coaxial-annotation

```
coaxial-annotation(first-i, first-j, second-i, second-j, stroke: auto)
```

Marks adjacent helices as coaxially stacked.

#coaxial-annotations

```
coaxial-annotations(result, stroke: auto)
```

Converts selected coaxial stacks in an energy result to annotations.

#label-annotation

```
label-annotation(at, body, dx: auto, dy: auto, anchor: "auto", distance: 13pt,  
leader: true, leader-stroke: auto, leader-bend: 0.0, leader-start-gap: 1pt,  
leader-end-gap: 1pt, fill: auto, size: auto, width: auto, text-align: left, box-  
fill: none, box-stroke: none, box-inset: 0pt, box-radius: 0pt)
```

Attaches a measured free-form label to a nucleotide. Offsets, leader geometry, wrapping width, and label box styling are independently adjustable.

#strand-label

```
strand-label(strand, body, at: "start", dx: auto, dy: auto, anchor: "auto",  
distance: 13pt, leader: true, leader-stroke: auto, leader-bend: 0.0, leader-  
start-gap: 1pt, leader-end-gap: 1pt, fill: auto, size: auto, width: auto, text-  
align: left, box-fill: none, box-stroke: none, box-inset: 0pt, box-radius: 0pt)
```

Labels either end of one strand with the same placement controls as a nucleotide label.

#numbering-style

```
numbering-style(every: 10, positions: (), per-strand: false, show-first: false,  
show-last: false, size: 5.3pt, fill: auto)
```

Configures nucleotide numbering.

#color-scale

```
color-scale(minimum: 0.0, maximum: 1.0, colors: (...), missing: luma(88%), label:  
none)
```

Defines a continuous value-to-color mapping.

#wcag-contrast-ratio

```
wcag-contrast-ratio(foreground, background, background-behind: auto)
```

Returns the WCAG 2.2 sRGB contrast ratio. Foreground and effective background must be opaque solid colors.

#wcag-text-fill

```
wcag-text-fill(background, level: "aa", background-behind: auto, on-failure:  
"error")
```

Chooses black or white for maximum WCAG contrast. AA requires 4.5:1; AAA requires 7:1 and fails unless on-failure: "best" is explicitly selected.

#value-annotations

```
value-annotations(values, scale: color-scale(), positions: auto, legend: auto)
```

XIV API reference: annotations and plots

Converts values to a metadata-bearing annotation track. legend may disable the metadata or provide width, height, ticks, text-size, orientation, reverse, and format options.

#color-legend

```
color-legend(scale, width: 3.2cm, height: 6pt, ticks: 3, text-size: auto,
orientation: "horizontal", reverse: false, format: auto, theme: plot-theme(),
inset: auto, fill: auto, stroke: auto, radius: auto)
```

Draws the exact scale legend horizontally or vertically. A formatter function receives each numerical tick value. Panel geometry and paint can inherit from the theme or be overridden directly; for example, stroke: none removes the legend frame.

#with-legend

```
with-legend(body, legend, position: "bottom", gutter: 6pt, style: auto)
```

Composes a manually constructed or shared legend without rasterization. style accepts the complete legend-style placement model; component drawings with metadata-bearing tracks should use legend: false.

#reactivity-annotations

```
reactivity-annotations(values, low: 0.0, high: 1.0, missing-fill: luma(88%),
scale: auto, legend: auto)
```

Converts probing values to a metadata-bearing color track.

#accessibility-annotations

```
accessibility-annotations(result, window-length: 1, scale: auto, legend: auto)
```

Converts accessibility results to a metadata-bearing color track.

#local-accessibility-annotations

```
local-accessibility-annotations(result, window-length: 1, scale: auto, legend:
auto)
```

Converts local accessibility data to a metadata-bearing color track.

#entropy-annotations

```
entropy-annotations(result, high: auto, missing-fill: luma(88%), scale: auto,
legend: auto)
```

Converts positional entropy to a metadata-bearing color track.

#ensemble-defect

```
ensemble-defect(sequence, target, result)
```

XIV API reference: annotations and plots

Computes normalized nucleotide ensemble defect from an existing analysis result.

#topology-annotations

```
topology-annotations(topology, palette: (...), thickness: 1.15pt)
```

Colors crossing components from a fatgraph topology result.

#plot-theme

```
plot-theme(text-size: 6.5pt, label-size: 7pt, text-fill: ..., label-fill: ...,  
grid-stroke: ..., minor-grid-stroke: ..., frame-stroke: ..., tick-stroke: ...,  
minor-tick-stroke: ..., background: none, legend-fill: white, legend-stroke: ...,  
legend-radius: 2pt)
```

Defines independent visual properties for plot frames, grids, ticks, text, backgrounds, and legend panels.

#axis-style

```
axis-style(domain: auto, ticks: auto, tick-step: auto, minor-ticks: none, minor-  
tick-step: none, tick-count: 5, format: auto, label: auto, mode: "linear", base:  
10, grid: "major", show-line: true, show-ticks: true, show-labels: true)
```

Configures one ascending or descending linear/logarithmic axis. Tick entries may be numbers or (value, label) pairs; format accepts auto, none, "scientific" or a function.

#plot-layout

```
plot-layout(padding: auto, aspect: auto, frame: true, clip: true)
```

Configures data-viewport padding, width-to-height aspect, frame composition, and clipping.

#legend-style

```
legend-style(position: "bottom", anchor: auto, offset: (0pt, 0pt), direction:  
"row", columns: auto, max-columns: auto, gutter: 6pt, item-gap: auto, row-gap:  
3pt, width: auto, inset: auto, fill: auto, stroke: auto, radius: auto)
```

Configures four outer sides, nine inner anchors, an explicit (x, y) viewport ratio, or none, together with flow and panel styling.

#legend-panel

```
legend-panel(body, style: legend-style(), theme: plot-theme())
```

Wraps arbitrary Typst content in a measured legend panel.

#legend-item

```
legend-item(label, kind: "line", stroke: ..., fill: black, radius: 2.6pt,  
preview: none)
```

XIV API reference: annotations and plots

Constructs a line, circle, square, or custom-content entry for `plot-legend`.

#`plot-legend`

```
plot-legend(items, style: legend-style(), theme: plot-theme())
```

Builds a standalone categorical legend with explicit columns and row/column flow.

#`place-legend`

```
place-legend(body, legend, style: legend-style())
```

Places a standalone legend around or over arbitrary composed content.

#`structure-difference`

```
structure-difference(sequence, reference, alternative, execution: execution-policy())
```

Classifies shared and unique pairs.

#`compare-structures`

```
compare-structures(sequence, reference, alternative, method: "naview", common-color: ..., reference-color: ..., alternative-color: ..., legend: true, legend-position: "bottom", legend-theme: plot-theme(), execution: execution-policy(), ..render-arguments)
```

Overlays two pair sets on reference coordinates. `legend` accepts a Boolean or `legend-style`.

#`dot-plot`

```
dot-plot(sequence, probabilities: auto, comparison: none, reference-structure: auto, width: 8cm, height: auto, threshold: 0.01, model: analysis-model(), constraints: none, execution: execution-policy(), scale: color-scale(), comparison-scale: color-scale(), legend: true, legend-position: "bottom", x-label: ..., y-label: ..., x-axis: axis-style(), y-axis: axis-style(), layout: plot-layout(), theme: plot-theme(), grid-stroke: auto, frame-stroke: auto)
```

Draws pair probabilities in an exact square data viewport and optionally compares two ensembles. Axes, viewport, and legend placement are independent.

#`mountain-profile`

```
mountain-profile(sequence, probabilities: auto, reference-structures: auto, model: analysis-model(), constraints: none, execution: execution-policy())
```

Returns the expected and discrete mountain values used for drawing.

#`mountain-plot`

```
mountain-plot(sequence, probabilities: auto, reference-structures: auto, width:
10cm, height: 3.8cm, model: analysis-model(), constraints: none, execution:
execution-policy(), stroke: ..., reference-strokes: (...), y-ticks: 4, legend:
true, legend-position: "bottom", x-label: ..., y-label: ..., expected-axes: ("x",
"y"), x-axis: axis-style(), y-axis: axis-style(), x2-axis: none, y2-axis: none,
layout: plot-layout(), theme: plot-theme())
```

Draws expected and discrete mountain profiles. Reference dictionaries select primary or secondary axes through axes.

Part XV

Errors, reproducibility, and performance

XV.1 Error handling

`request` panics with a stable error code and message. `try-request` returns the same information as `data`:

```
#let response = try-request(  
  "validate",  
  (sequence: "GGG", structure: "(("."),  
)  
  
#if not response.ok {  
  [#response.error.code: #response.error.message]  
}
```

analysis_failed: unmatched opening bracket '(' at structure position 2

Common categories include invalid input, unsupported options, constraint conflicts, and resource limits. Do not parse prose to branch on failures; use `error.code`.

XV.2 Determinism

Layouts and deterministic decoders are stable for the same package version and input. Sampling is reproducible when seed is fixed. Model and parameter fingerprints are included in responses. For archival work, record the package version, model dictionary, constraints, operation options, and input sequence/structure.

XV.3 Performance guidance

- Prefer one `analyze` call when MFE, probabilities, centroid, and MEA are all needed.
- Reuse its response for rendering and plots.
- Use `fold` when only an MFE structure is needed.
- Use local analysis for long targets when a global ensemble is not scientifically required.
- Keep `allow-expensive` local to a deliberately bounded call.
- For long figures, leave `detail: auto` enabled unless every nucleotide label is essential.

Part XVI

Validation and reproducible export checklist

The release validation suite includes 24 diverse families selected from Rfam [33] and 24 published pseudoknot records. It checks numerical invariants, external reference values, six structure layouts, all result-to-render paths, every inner and outer legend anchor, explicit legend coordinates, linear/logarithmic/reversed/secondary axes, vector-only PDFs, clipping and blank pages, semantic metadata, and pixel-exact golden images.

Before submitting a figure:

- state package version and thermodynamic model in methods or supplementary material;
- report temperature, salt, dangle model, constraints, and probing conversion;
- use a legend for every continuous annotation scale;
- distinguish predicted, comparative, and experimentally constrained structures;
- inspect label collisions after the final page size is chosen;
- retain vector PDF/SVG output instead of taking a screenshot;
- record seeds for sampled structures;
- avoid implying that a pseudoknot decoder and a full thermodynamic ensemble are interchangeable.

Repository validation details are maintained in `docs/VALIDATION.md`. The model and recurrence boundary is documented in `docs/MODEL.md`.

Part XVII

License, provenance, and citation

Ribon is distributed under GPL-2.0-only. The bundled Turner 2004 RNA and Mathews 2004 DNA tables [9] are generated from the official RNAstructure 6.6 distribution.⁴

When citing Ribon, cite the archived release of the package and the specific thermodynamic, layout, probing, decoding, or pseudoknot model used in the reported analysis. The bibliography below is maintained in `docs/references.bib`. Web-only standards and source archives are cited in footnotes at the claims they support.

⁴University of Rochester, “RNAstructure,” <https://rna2.urmc.rochester.edu/RNAstructure.html>. Exact archive hashes and licensing details are recorded in the package `NOTICE.md`.

References

1. Brucoleri RE, Heinrich G. An Improved Algorithm for Nucleic Acid Secondary Structure Display. *Computer Applications in the Biosciences*. 1988;4: 167–173. doi:[10.1093/bioinformatics/4.1.167](https://doi.org/10.1093/bioinformatics/4.1.167)
2. Wiegreffe D, Alexander D, Stadler PF, Zeckzer D. RNApuzzler: Efficient Outerplanar Drawing of RNA-Secondary Structures. *Bioinformatics*. 2019;35: 1342–1349. doi:[10.1093/bioinformatics/bty817](https://doi.org/10.1093/bioinformatics/bty817)
3. Lorenz R, Bernhart SH, Siederdisen C Höner zu, Tafer H, Flamm C, Stadler PF, et al. ViennaRNA Package 2.0. *Algorithms for Molecular Biology*. 2011;6: 26. doi:[10.1186/1748-7188-6-26](https://doi.org/10.1186/1748-7188-6-26)
4. Darty K, Denise A, Ponty Y. VARNA: Interactive Drawing and Editing of the RNA Secondary Structure. *Bioinformatics*. 2009;25: 1974–1975. doi:[10.1093/bioinformatics/btp250](https://doi.org/10.1093/bioinformatics/btp250)
5. Zuker M, Stiegler P. Optimal Computer Folding of Large RNA Sequences Using Thermodynamics and Auxiliary Information. *Nucleic Acids Research*. 1981;9: 133–148. doi:[10.1093/nar/9.1.133](https://doi.org/10.1093/nar/9.1.133)
6. McCaskill JS. The Equilibrium Partition Function and Base Pair Binding Probabilities for RNA Secondary Structure. *Biopolymers*. 1990;29: 1105–1119. doi:[10.1002/bip.360290621](https://doi.org/10.1002/bip.360290621)
7. Hamada M, Kiryu H, Sato K, Mituyama T, Asai K. Prediction of RNA Secondary Structure Using Generalized Centroid Estimators. *Bioinformatics*. 2009;25: 465–473. doi:[10.1093/bioinformatics/btn601](https://doi.org/10.1093/bioinformatics/btn601)
8. Lu ZJ, Gloor JW, Mathews DH. Improved RNA Secondary Structure Prediction by Maximizing Expected Pair Accuracy. *RNA*. 2009;15: 1805–1813. doi:[10.1261/rna.1643609](https://doi.org/10.1261/rna.1643609)
9. Mathews DH, Disney MD, Childs JL, Schroeder SJ, Zuker M, Turner DH. Incorporating Chemical Modification Constraints into a Dynamic Programming Algorithm for Prediction of RNA Secondary Structure. *Proceedings of the National Academy of Sciences*. 2004;101: 7287–7292. doi:[10.1073/pnas.0401799101](https://doi.org/10.1073/pnas.0401799101)
10. Yao H-T, Lorenz R, Hofacker IL, Stadler PF. Mono-Valent Salt Corrections for RNA Secondary Structures in the ViennaRNA Package. *Algorithms for Molecular Biology*. 2023;18: 8. doi:[10.1186/s13015-023-00236-0](https://doi.org/10.1186/s13015-023-00236-0)
11. Deigan KE, Li TW, Mathews DH, Weeks KM. Accurate SHAPE-Directed RNA Structure Determination. *Proceedings of the National Academy of Sciences*. 2009;106: 97–102. doi:[10.1073/pnas.0806929106](https://doi.org/10.1073/pnas.0806929106)
12. Zarringhalam K, Meyer MM, Dotu I, Chuang JH, Clote P. Integrating Chemical Footprinting Data into RNA Secondary Structure Prediction. *PLOS ONE*. 2012;7: e45160. doi:[10.1371/journal.pone.0045160](https://doi.org/10.1371/journal.pone.0045160)
13. Wuchty S, Fontana W, Hofacker IL, Schuster P. Complete Suboptimal Folding of RNA and the Stability of Secondary Structures. *Biopolymers*. 1999;49: 145–165. doi:[10.1002/\(SICI\)1097-0282\(199902\)49:2<145::AID-BIP4>3.0.CO;2-G](https://doi.org/10.1002/(SICI)1097-0282(199902)49:2<145::AID-BIP4>3.0.CO;2-G)
14. Bernhart SH, Hofacker IL, Stadler PF. Local RNA Base Pairing Probabilities in Large Sequences. *Bioinformatics*. 2006;22: 614–615. doi:[10.1093/bioinformatics/btk014](https://doi.org/10.1093/bioinformatics/btk014)
15. Bernhart SH, Tafer H, Mückstein U, Flamm C, Stadler PF, Hofacker IL. Partition Function and Base Pairing Probabilities of RNA Heterodimers. *Algorithms for Molecular Biology*. 2006;1: 3. doi:[10.1186/1748-7188-1-3](https://doi.org/10.1186/1748-7188-1-3)
16. Hogeweg P, Hesper B. Energy Directed Folding of RNA Sequences. *Nucleic Acids Research*. 1984;12: 67–74. doi:[10.1093/nar/12.1Part1.67](https://doi.org/10.1093/nar/12.1Part1.67)
17. Kierzek E, Zhang X, Watson RM, Kennedy SD, Szabat M, Kierzek R, et al. Secondary Structure Prediction for RNA Sequences Including N6-Methyladenosine. *Nature Communications*. 2022;13: 1271. doi:[10.1038/s41467-022-28817-4](https://doi.org/10.1038/s41467-022-28817-4)
18. Hudson GA, Bloomingdale RJ, Znosko BM. Thermodynamic Contribution and Nearest-Neighbor Parameters of Pseudouridine–Adenosine Base Pairs in Oligoribonucleotides. *RNA*. 2013;19: 1474–1482. doi:[10.1261/rna.039610.113](https://doi.org/10.1261/rna.039610.113)
19. Wright DJ, Force CR, Znosko BM. Stability of RNA Duplexes Containing Inosine–Cytosine Pairs. *Nucleic Acids Research*. 2018;46: 12099–12108. doi:[10.1093/nar/gky907](https://doi.org/10.1093/nar/gky907)

References

20. Richardson KE, Znosko BM. Nearest-Neighbor Parameters for 7-Deaza-Adenosine–Uridine Base Pairs in RNA Duplexes. *RNA*. 2016;22: 934–942. doi:[10.1261/rna.055277.115](https://doi.org/10.1261/rna.055277.115)
21. Jolley EA, Znosko BM. The Loss of a Hydrogen Bond: Thermodynamic Contributions of a Non-Standard Nucleotide. *Nucleic Acids Research*. 2017;45: 1479–1487. doi:[10.1093/nar/gkw830](https://doi.org/10.1093/nar/gkw830)
22. Dalluge JJ, Hashizume T, Sopchik AE, McCloskey JA, Davis DR. Conformational Flexibility in RNA: The Role of Dihydrouridine. *Nucleic Acids Research*. 1996;24: 1073–1079. doi:[10.1093/nar/24.6.1073](https://doi.org/10.1093/nar/24.6.1073)
23. Varenky Y, Spicher T, Hofacker IL, Lorenz R. Modified RNAs and Predictions with the ViennaRNA Package. *Bioinformatics*. 2023;39: btad696. doi:[10.1093/bioinformatics/btad696](https://doi.org/10.1093/bioinformatics/btad696)
24. Lorenz R, Bernhart SH, Qin J, Siederdisen C Höner zu, Tanzer A, Amman F, et al. 2D Meets 4G: G-Quadruplexes in RNA Secondary Structure Prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*. 2013;10: 832–844. doi:[10.1109/TCBB.2013.7](https://doi.org/10.1109/TCBB.2013.7)
25. Hofacker IL, Fekete M, Stadler PF. Secondary Structure Prediction for Aligned RNA Sequences. *Journal of Molecular Biology*. 2002;319: 1059–1066. doi:[10.1016/S0022-2836\(02\)00308-X](https://doi.org/10.1016/S0022-2836(02)00308-X)
26. Bellaousov S, Mathews DH. ProbKnot: Fast Prediction of RNA Secondary Structure Including Pseudoknots. *RNA*. 2010;16: 1870–1880. doi:[10.1261/rna.2125310](https://doi.org/10.1261/rna.2125310)
27. Kuhn HW. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*. 1955;2: 83–97. doi:[10.1002/nav.3800020109](https://doi.org/10.1002/nav.3800020109)
28. Jabbari H, Condon A, Zhao S. Novel and Efficient RNA Secondary Structure Prediction Using Hierarchical Folding. *Journal of Computational Biology*. 2008;15: 139–163. doi:[10.1089/cmb.2007.0198](https://doi.org/10.1089/cmb.2007.0198)
29. Andronescu MS, Pop C, Condon AE. Improved Free Energy Parameters for RNA Pseudoknotted Secondary Structure Prediction. *RNA*. 2010;16: 26–42. doi:[10.1261/rna.1689910](https://doi.org/10.1261/rna.1689910)
30. Gray M, Trinity L, Stege U, Jabbari H. CParty: Hierarchically Constrained Partition Function of RNA Pseudoknots. *Bioinformatics*. 2025;41: btae748. doi:[10.1093/bioinformatics/btae748](https://doi.org/10.1093/bioinformatics/btae748)
31. Bon M, Vernizzi G, Orland H, Zee A. Topological Classification of RNA Structures. *Journal of Molecular Biology*. 2008;379: 900–911. doi:[10.1016/j.jmb.2008.04.033](https://doi.org/10.1016/j.jmb.2008.04.033)
32. Dijkstra EW. A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*. 1959;1: 269–271. doi:[10.1007/BF01386390](https://doi.org/10.1007/BF01386390)
33. Kalvari I, Nawrocki EP, Ontiveros-Palacios N, Argasinska J, Lamkiewicz K, Marz M, et al. Rfam 14: Expanded Coverage of Metagenomic, Viral and microRNA Families. *Nucleic Acids Research*. 2021;49: D192–D200. doi:[10.1093/nar/gkaa1047](https://doi.org/10.1093/nar/gkaa1047)

Part XVIII

Index

A

[#accessibility](#) 37
[#accessibility-annotations..](#)
45
[#accessibility-window](#) 37
[#analysis-model](#) 33
[#analyze](#) 36
[#axis-style](#) 46

B

[#base-annotation](#) 43

C

[#capabilities](#) 36
[#circular](#) 37
[#coaxial-annotation](#) 43
[#coaxial-annotations](#) 43
[#cofold](#) 37
[#color-legend](#) 45
[#color-scale](#) 44
[#comparative](#) 38
[#comparative-options](#) 38
[#compare-structures](#) 47
[#conditional-density2](#) 39
[#conditional-density2-](#)
[options](#) 39
[#conditional-density2-](#)
[sample](#) 39
[#conditional-density2-](#)
[suboptimal](#) 39
[#constraint-pair](#) 35
[#custom-model](#) 33

D

[#data](#) 34
[#dna-model](#) 33
[#dot-plot](#) 47
[#draw](#) 41
[#duplex](#) 37

E

[#element-annotations](#) 41
[#ensemble-defect](#) 45
[#entropy-annotations](#) 45
[#evaluate](#) 36
[#evaluate-conditional-](#)
[density2](#) 39
[#evaluate-pseudoknot](#) 39
[#execution-policy](#) 33

F

[#fatgraph-topology](#) 39
[#fold](#) 36
[#folding-constraints](#) 35

G

[#gquad](#) 38

H

[#highlight](#) 43
[#highlight-positions](#) 43

I

[#interaction-annotation](#) . 43
[#inverse-design](#) 40

L

[#label-annotation](#) 44
[#landscape](#) 40
[#layout](#) 41
[#legend-item](#) 46
[#legend-panel](#) 46
[#legend-style](#) 46
[#ligand](#) 40
[#ligand-motif](#) 40
[#local](#) 37
[#local-accessibility-](#)
[annotations](#) 45

M

[#modified](#) 38
[#modified-base](#) 38
[#mountain-plot](#) 47
[#mountain-profile](#) 47

N

[#numbering-style](#) 44

P

[#pair-annotation](#) 43
[#pair-energy](#) 35
[#parameters](#) 36
[#place-legend](#) 47
[#plot-layout](#) 46
[#plot-legend](#) 47
[#plot-theme](#) 46
[#position-energy](#) 35
[#probing-data](#) 35
[#pseudoknot](#) 38
[#pseudoknot-options](#) 38

R

[#reactivity-annotations](#) . 45
[#render](#) 41
[#render-scene](#) 41
[#request](#) 33

S

[#sample](#) 36
[#soft-constraints](#) 35
[#strand-label](#) 44
[#structure-difference](#) 47
[#structure-elements](#) 41
[#suboptimal](#) 36

T

[#thermodynamic-parameter-](#)
[overrides](#) 33
[#topology-annotations](#) 46
[#try-request](#) 33

V

#validate	36
#value-annotations	44

W

#wcag-contrast-ratio	44
#wcag-text-fill	44
#with-legend	45